



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**

Faculty Preparatory Program

Course Name: Machine Learning Laboratory

Course Code: AL23421

Academic Year: II (2025-2026)

Semester: III (EVEN)

LAB MANUAL

Prepared By

**Course
Coordinator**

**Approve by
HOD**

Dean



VISION OF THE DEPARTMENT

To encourage more employability, innovative research, and entrepreneurial thinking by developing emerging technology solutions for the benefit of industry and society through effective teaching and learning processes.

MISSION OF THE DEPARTMENT

- ❖ Providing high-quality, value-added technical education and producing technology professionals who can think creatively and inspire others.
- ❖ Applying rational thought to design and create cutting-edge products in collaboration with industry stakeholders in order to meet global standards and demands.
- ❖ Improving essential skills in Artificial Intelligence and Machine Learning.



PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

Graduates can

- ❖ To excel in society, students will significantly advance their knowledge of the principles of engineering, science, and technology.
- ❖ To pursue higher education in an AI environment to address issues in the real world.
- ❖ To receive training in creating, developing, and quality assurance knowledge of AI and ML technologies in order to produce solutions to challenges encountered in the real world.
- ❖ To provide students with the knowledge and skills necessary to use computational tools and AI and ML effectively.
- ❖ To encourage graduates with advancing the knowledge of AI and ML approaches among entrepreneurs and researchers.



PROGRAM OUTCOMES (POs)

1. Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of the information to provide valid conclusions.

5. Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts,

and demonstrate the knowledge of, and need for sustainable development.

8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-Long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



PROGRAM SPECIFIC OUTCOMES (PSOs)

The students will be able to

1. Create and implement optimized solutions for computationally demanding applications.
2. Use Artificial Intelligence and Machine Learning Tools and Techniques to solve multidisciplinary problems
3. Create AI and Machine Learning models on data to facilitate better decision-making in societal issues.



AL23421

MACHINE LEARNING LABORATORY

Academic Year: 2025- 26

Year/ Semester: II Year / III Semester (Even)

Course Objectives

The student should be made to:

- To understand the data sets and apply suitable algorithms for selecting the appropriate features for analysis.
- To learn to implement supervised machine learning algorithms on standard datasets and evaluate the performance.
- To experiment the unsupervised machine learning algorithms on standard datasets and evaluate the performance.
- To build the graph-based learning models for standard data sets.
- To compare the performance of different ML algorithms and select the suitable one based on the application.

Course Outcomes:

At the end of the course, students will be able to

CO1: Apply suitable algorithms for selecting the appropriate features for analysis.

CO2: Implement supervised machine learning algorithms on standard datasets and evaluate the performance.

CO3: Apply unsupervised machine learning algorithms on standard datasets and evaluate the performance.

CO4: Build the graph-based learning models for standard data sets.

CO5: Assess and compare the performance of different ML algorithms and select the suitable one based on the application.

CO's – PO's and PSO's Mapping

CO	PO												PSO		
	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
1	2	2	2	1	-	-	-	-	1	2	3	3	3	2	1
2	2	1	1	3	2	-	-	-	3	2	3	2	3	1	1
3	2	2	1	1	2	-	-	-	1	1	1	1	2	3	3
4	2	2	3	3	2	-	-	-	1	2	1	1	1	2	2
5	2	2	3	1	2	-	-	-	3	1	1	1	2	1	2

1 - low, 2 - medium, 3 - high, - no correlation

SYLLABUS

LIST OF EXPERIMENTS

1. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.
2. Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.
3. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.
4. Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file and compute the accuracy with a few test data sets.
5. Implement naïve Bayesian Classifier model to classify a set of documents and measure the accuracy, precision, and recall.
6. Write a program to construct a Bayesian network to diagnose CORONA infection using standard WHO Data Set.
7. Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same data set for clustering using the k-Means algorithm. Compare the results of these two algorithms.
8. Write a program to implement k-Nearest Neighbour algorithm to classify the iris data set. Print both correct and wrong predictions.
9. Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select an appropriate data set for your experiment and draw graphs.

TOTAL: 30 PERIODS

SOFTWARE SPECIFICATIONS:

Programming Language: Python

Version: 3.11.8

**Integrated Development Environment: Microsoft
Visual studio**

LIST OF EXPERIMENTS

S.NO	TASK NAME
1	Implementation of candidate –elimination algorithm
2	Implementation of decision tree in id3 algorithm
3	Implementation of back propagation algorithm to build an artificial neural network
4	Implementation of naïve bayesian classifier for a sample training dataset and to compute accuracy
5	Implementation of naïve bayesian classifier model to classify a set of documents and to measure the accuracy, precision, and recall
6	Construction of a bayesian network to diagnose corona infection using standard who data set
7	Comparison of clustering in em algorithm and k-means algorithm using the same data sets
8	Implementation of k-nearest neighbour algorithm to classify the iris data set
9	Implementation of the non-parametric locally weighted regression algorithm in order to fit data points

EX. NO: 1

IMPLEMENTATION OF CANDIDATE –ELIMINATION ALGORITHM

AIM:

To implement and demonstrate the Candidate-Elimination algorithm, for a given set of training data examples stored in a .CSV file, to output a description of the set of all hypotheses consistent with the training examples.

ALGORITHM:

1. Load Data set.
2. Initialize General Hypothesis and Specific Hypothesis.
3. For each training example
4. If example is positive example
 - if attribute_value == hypothesis_value:
Do nothing
 - else:
replace attribute value with '?' (Basically generalizing it)
5. If example is Negative example
 - Make generalize hypothesis more specific.

PROGRAM:

dataset.csv

outlook	temperatu	humidity	wind	answer
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rain	mild	high	weak	yes
rain	cool	normal	weak	yes
rain	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rain	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rain	mild	high	strong	no

```

import numpy as np
import pandas as pd
# Loading Data from a CSV File
data = pd.DataFrame(data=pd.read_csv('E:\BALA\AI\Lab programs\pgms\dataset.csv'))
print(data)

```

	outlook	temperature	humidity	wind	answer
0	sunny	hot	high	weak	no
1	sunny	hot	high	strong	no
2	overcast	hot	high	weak	yes
3	rain	mild	high	weak	yes
4	rain	cool	normal	weak	yes
5	rain	cool	normal	strong	no
6	overcast	cool	normal	strong	yes
7	sunny	mild	high	weak	no
8	sunny	cool	normal	weak	yes
9	rain	mild	normal	weak	yes
10	sunny	mild	normal	strong	yes
11	overcast	mild	high	strong	yes
12	overcast	hot	normal	weak	yes
13	rain	mild	high	strong	no

```

# Separating concept features from Target

```

```

concepts = np.array(data.iloc[:,0:-1])
print(concepts)

```

```

[['sunny' 'hot' 'high' 'weak']
 ['sunny' 'hot' 'high' 'strong']
 ['overcast' 'hot' 'high' 'weak']
 ['rain' 'mild' 'high' 'weak']
 ['rain' 'cool' 'normal' 'weak']
 ['rain' 'cool' 'normal' 'strong']
 ['overcast' 'cool' 'normal' 'strong']
 ['sunny' 'mild' 'high' 'weak']
 ['sunny' 'cool' 'normal' 'weak']
 ['rain' 'mild' 'normal' 'weak']
 ['sunny' 'mild' 'normal' 'strong']
 ['overcast' 'mild' 'high' 'strong']
 ['overcast' 'hot' 'normal' 'weak']
 ['rain' 'mild' 'high' 'strong']]

```

```

# Isolating target into a separate DataFrame

```

```

# copying last column to target array

```

```

target = np.array(data.iloc[:, -1])
print(target)

```

```
['no' 'no' 'yes' 'yes' 'yes' 'no' 'yes' 'no' 'yes' 'yes' 'yes' 'yes' 'yes'
 'no']
```

```
def learn(concepts, target):
```

```
    """
```

```
    learn() function implements the learning method of the Candidate elimination algorithm.
```

```
    Arguments:
```

```
        concepts - a data frame with all the features
```

```
        target - a data frame with corresponding output values
```

```
    """
```

```
    # Initialise S0 with the first instance from concepts
```

```
    # .copy() makes sure a new list is created instead of just pointing to the same memory location
```

```
    specific_h = concepts[0].copy()
```

```
    print("\nInitialization of specific_h and general_h")
```

```
    print(specific_h)
```

```
    #h=["#" for i in range(0,5)]
```

```
    #print(h)
```

```
    general_h = [["?" for i in range(len(specific_h))] for i in range(len(specific_h))]
```

```
    print(general_h)
```

```
    # The learning iterations
```

```
    for i, h in enumerate(concepts):
```

```
        # Checking if the hypothesis has a positive target
```

```
        if target[i] == "Yes":
```

```
            for x in range(len(specific_h)):
```

```
                # Change values in S & G only if values change
```

```
                if h[x] != specific_h[x]:
```

```
                    specific_h[x] = '?'
```

```
                    general_h[x][x] = '?'
```

```
        # Checking if the hypothesis has a positive target
```

```

if target[i] == "No":
    for x in range(len(specific_h)):
        # For negative hypothesis change values only in G
        if h[x] != specific_h[x]:
            general_h[x][x] = specific_h[x]
        else:
            general_h[x][x] = '?'
    print("\nSteps of Candidate Elimination Algorithm",i+1)
    print(specific_h)
    print(general_h)
# find indices where we have empty rows, meaning those that are unchanged
indices = [i for i, val in enumerate(general_h) if val == ['?', '?', '?', '?', '?', '?']]
for i in indices:
    # remove those rows from general_h
    general_h.remove(['?', '?', '?', '?', '?', '?'])
# Return final values
return specific_h, general_h
s_final, g_final = learn(concepts, target)
print("\nFinal Specific_h:", s_final, sep="\n")
print("\nFinal General_h:", g_final, sep="\n")

```

OUTPUT:

```

Initialization of specific_h and general_h
['sunny' 'hot' 'high' 'weak']
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]

Steps of Candidate Elimination Algorithm 1
['sunny' 'hot' 'high' 'weak']
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]

Steps of Candidate Elimination Algorithm 2
['sunny' 'hot' 'high' 'weak']
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]

```

Steps of Candidate Elimination Algorithm 3

```
['sunny' 'hot' 'high' 'weak']  
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]
```

Steps of Candidate Elimination Algorithm 4

```
['sunny' 'hot' 'high' 'weak']  
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]
```

Steps of Candidate Elimination Algorithm 5

```
['sunny' 'hot' 'high' 'weak']  
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]
```

Steps of Candidate Elimination Algorithm 6

```
['sunny' 'hot' 'high' 'weak']  
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]
```

Steps of Candidate Elimination Algorithm 7

```
['sunny' 'hot' 'high' 'weak']  
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]
```

Steps of Candidate Elimination Algorithm 8

```
['sunny' 'hot' 'high' 'weak']  
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]
```

Steps of Candidate Elimination Algorithm 9

```
['sunny' 'hot' 'high' 'weak']  
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]
```

Steps of Candidate Elimination Algorithm 10

```
['sunny' 'hot' 'high' 'weak']  
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]
```

Steps of Candidate Elimination Algorithm 11

```
['sunny' 'hot' 'high' 'weak']  
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]
```

Steps of Candidate Elimination Algorithm 12

```
['sunny' 'hot' 'high' 'weak']  
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]
```

Steps of Candidate Elimination Algorithm 13

```
['sunny' 'hot' 'high' 'weak']  
[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]
```

Steps of Candidate Elimination Algorithm 14

['sunny' 'hot' 'high' 'weak']

[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]

Final Specific_h:

['sunny' 'hot' 'high' 'weak']

Final General_h:

[['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', '?']]

RESULT:

Thus the Candidate-Elimination algorithm, to test all the hypotheses with the training sets using python was executed and verified successfully.

EX. NO: 2

DATE :

IMPLEMENTATION OF DECISION TREE IN ID3 ALGORITHM

AIM:

To build Decision tree in ID3 algorithm to classify a new sample using python.

ALGORITHM:

1. Observe the dataset. Import the necessary basic python libraries.
2. Read the dataset.
3. Calculate the Entropy of the whole dataset.
4. Calculate the Entropy of the filtered dataset.
5. Calculate the Information gain for the feature(outlook).
6. Finding the most informative feature (feature with highest information gain).
7. Adding a node to the tree.
8. Perform ID3 algorithm and generate a tree.
9. Finding unique classes of the label.
10. Predicting from the tree.
11. Evaluating the test dataset.
12. Checking the test dataset.

PROGRAM:

```
import numpy as np
import math
import csv
def read_data(filename):
    with open(filename, 'r') as csvfile:
        datareader = csv.reader(csvfile, delimiter=',')
        headers = next(datareader)
        metadata = []
        traindata = []
        for name in headers:
            metadata.append(name)
        for row in datareader:
```

```

        traindata.append(row)
    return (metadata, traindata)

class Node:
    def __init__(self, attribute):
        self.attribute = attribute
        self.children = []
        self.answer = ""

    def __str__(self):
        return self.attribute

def subtables(data, col, delete):
    dict = {}
    items = np.unique(data[:, col])
    count = np.zeros((items.shape[0], 1), dtype=np.int32)
    for x in range(items.shape[0]):
        for y in range(data.shape[0]):
            if data[y, col] == items[x]:
                count[x] += 1
    for x in range(items.shape[0]):
        dict[items[x]] = np.empty((int(count[x]), data.shape[1]), dtype="|S32")
        pos = 0
        for y in range(data.shape[0]):
            if data[y, col] == items[x]:
                dict[items[x]][pos] = data[y]
                pos += 1
    if delete:
        dict[items[x]] = np.delete(dict[items[x]], col, 1)
    return items, dict

```

```

def entropy(S):
    items = np.unique(S)
    if items.size == 1:
        return 0
    counts = np.zeros((items.shape[0], 1))
    sums = 0
    for x in range(items.shape[0]):
        counts[x] = sum(S == items[x]) / (S.size * 1.0)
    for count in counts:
        sums += -1 * count * math.log(count, 2)
    return sums

def gain_ratio(data, col):
    items, dict = subtables(data, col, delete=False)
    total_size = data.shape[0]
    entropies = np.zeros((items.shape[0], 1))
    intrinsic = np.zeros((items.shape[0], 1))
    for x in range(items.shape[0]):
        ratio = dict[items[x]].shape[0]/(total_size * 1.0)
        entropies[x] = ratio * entropy(dict[items[x]][:-1])
        intrinsic[x] = ratio * math.log(ratio, 2)
    total_entropy = entropy(data[:, -1])
    iv = -1 * sum(intrinsic)
    for x in range(entropies.shape[0]):
        total_entropy -= entropies[x]
    return total_entropy / iv

def create_node(data, metadata):
    if (np.unique(data[:, -1])).shape[0] == 1:
        node = Node("")

```

```

        node.answer = np.unique(data[:, -1])[0]
        return node

gains = np.zeros((data.shape[1] - 1, 1))
for col in range(data.shape[1] - 1):
    gains[col] = gain_ratio(data, col)
split = np.argmax(gains)
node = Node(metadata[split])
metadata = np.delete(metadata, split, 0)
items, dict = subtables(data, split, delete=True)
for x in range(items.shape[0]):
    child = create_node(dict[items[x]], metadata)
    node.children.append((items[x], child))

return node

def empty(size):
    s = ""
    for x in range(size):
        s += " "
    return s

def print_tree(node, level):
    if node.answer != "":
        print(empty(level), node.answer)
        return
    print(empty(level), node.attribute)
    for value, n in node.children:
        print(empty(level + 1), value)
        print_tree(n, level + 2)

metadata, traindata = read_data("E:\BALA\AI\Lab programs\pgms\Tennisdata.csv")
data = np.array(traindata)

```

```
node = create_node(data, metadata)
print_tree(node, 0)
```

OUTPUT:

```
Outlook
  Overcast
    b'Yes'
  Rainy
    Windy
      b'FALSE'
      b'Yes'
      b'TRUE'
      b'No'
  Sunny
    Humidity
      b'High'
      b'No'
      b'Normal'
      b'Yes'
```

RESULT:

Thus the program to implement decision tree based ID3 algorithm using python was executed and verified successfully.

EX. NO: 3

DATE :

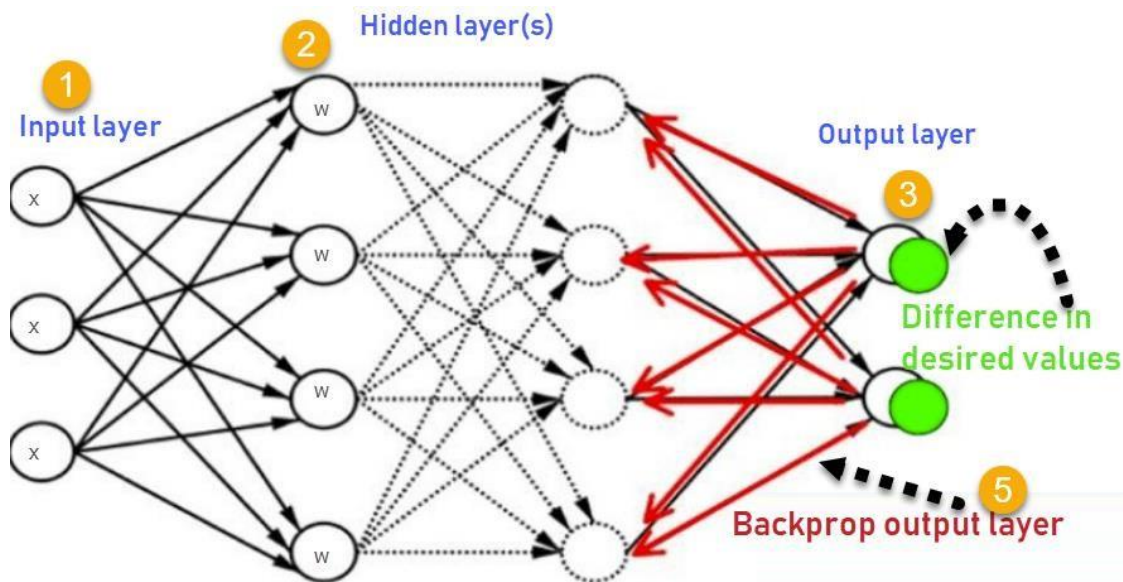
IMPLEMENTATION OF BACK PROPAGATION ALGORITHM TO BUILD AN ARTIFICIAL NEURAL NETWORK

AIM:

To implement the Back Propagation algorithm to build an Artificial Neural Network.

ALGORITHM:

1. Inputs X , arrive through the preconnected path.
2. Input is modeled using real weights W . The weights are usually randomly selected.
3. Calculate the output for every neuron from the input layer, to the hidden layers, to the output layer.
4. Calculate the error in the outputs
5. Travel back from the output layer to the hidden layer to adjust the weights such that the errors is decreased. Keep repeating the process until the desired output is achieved.



PROGRAM:

```
from math import exp
from random import seed
from random import random

# Initialize a network

def initialize_network(n_inputs, n_hidden, n_outputs):
```

```

network = list()

hidden_layer = [{ 'weights':[random() for i in range(n_inputs + 1)] } for i in
range(n_hidden)]

network.append(hidden_layer)

output_layer = [{ 'weights':[random() for i in range(n_hidden + 1)] } for i in
range(n_outputs)]

network.append(output_layer)

return network

# Calculate neuron activation for an input
def activate(weights, inputs):
    activation = weights[-1]
    for i in range(len(weights)-1):
        activation += weights[i] * inputs[i]
    return activation

# Transfer neuron activation
def transfer(activation):
    return 1.0 / (1.0 + exp(-activation))

# Forward propagate input to a network output
def forward_propagate(network, row):
    inputs = row
    for layer in network:
        new_inputs = []
        for neuron in layer:
            activation = activate(neuron['weights'], inputs)
            neuron['output'] = transfer(activation)
            new_inputs.append(neuron['output'])
        inputs = new_inputs
    return inputs

# Calculate the derivative of an neuron output

```

```

def transfer_derivative(output):
    return output * (1.0 - output)

# Backpropagate error and store in neurons
def backward_propagate_error(network, expected):
    for i in reversed(range(len(network))):
        layer = network[i]
        errors = list()
        if i != len(network)-1:
            for j in range(len(layer)):
                error = 0.0
                for neuron in network[i + 1]:
                    error += (neuron['weights'][j] * neuron['delta'])
                errors.append(error)
        else:
            for j in range(len(layer)):
                neuron = layer[j]
                errors.append(neuron['output'] - expected[j])
            for j in range(len(layer)):
                neuron = layer[j]
                neuron['delta'] = errors[j] * transfer_derivative(neuron['output'])

# Update network weights with error
def update_weights(network, row, l_rate):
    for i in range(len(network)):
        inputs = row[:-1]
        if i != 0:
            inputs = [neuron['output'] for neuron in network[i - 1]]
        for neuron in network[i]:
            for j in range(len(inputs)):

```



```

        neuron['weights'][j] -= l_rate * neuron['delta'] * inputs[j]

        neuron['weights'][-1] -= l_rate * neuron['delta']

# Train a network for a fixed number of epochs
def train_network(network, train, l_rate, n_epoch, n_outputs):
    for epoch in range(n_epoch):
        sum_error = 0
        for row in train:
            outputs = forward_propagate(network, row)
            expected = [0 for i in range(n_outputs)]
            expected[row[-1]] = 1

            sum_error += sum([(expected[i]-outputs[i])**2 for i in
range(len(expected))])
            backward_propagate_error(network, expected)
            update_weights(network, row, l_rate)

        print('>epoch=%d, lrate=%.3f, error=%.3f % (epoch, l_rate, sum_error))

# Test training backprop algorithm
seed(1)
dataset = [[2.7810836,2.550537003,0],
            [1.465489372,2.362125076,0],
            [3.396561688,4.400293529,0],
            [1.38807019,1.850220317,0],
            [3.06407232,3.005305973,0],
            [7.627531214,2.759262235,1],
            [5.332441248,2.088626775,1],
            [6.922596716,1.77106367,1],
            [8.675418651,-0.242068655,1],
            [7.673756466,3.508563011,1]]

n_inputs = len(dataset[0]) - 1
n_outputs = len(set([row[-1] for row in dataset]))

```

```

network = initialize_network(n_inputs, 2, n_outputs)
train_network(network, dataset, 0.5, 20, n_outputs)

for layer in network:
    print(layer)

```

OUTPUT:

```

>epoch=0, lrate=0.500, error=6.350
>epoch=1, lrate=0.500, error=5.531
>epoch=2, lrate=0.500, error=5.221
>epoch=3, lrate=0.500, error=4.951
>epoch=4, lrate=0.500, error=4.519
>epoch=5, lrate=0.500, error=4.173
>epoch=6, lrate=0.500, error=3.835
>epoch=7, lrate=0.500, error=3.506
>epoch=8, lrate=0.500, error=3.192
>epoch=9, lrate=0.500, error=2.898
>epoch=10, lrate=0.500, error=2.626
>epoch=11, lrate=0.500, error=2.377
>epoch=12, lrate=0.500, error=2.153
>epoch=13, lrate=0.500, error=1.953
>epoch=14, lrate=0.500, error=1.774
>epoch=15, lrate=0.500, error=1.614
>epoch=16, lrate=0.500, error=1.472
>epoch=17, lrate=0.500, error=1.346
>epoch=18, lrate=0.500, error=1.233
>epoch=19, lrate=0.500, error=1.132
[{'weights': [-1.4688375095432327, 1.850887325439514, 1.0858178629550297], 'output': 0.029
980305604426185, 'delta': 0.0059546604162323625}, {'weights': [0.37711098142462157, -0.06
25909894552989, 0.2765123702642716], 'output': 0.9456229000211323, 'delta': -0.0026279652
850863837}]
[{'weights': [2.515394649397849, -0.3391927502445985, -0.9671565426390275], 'output': 0.23
648794202357587, 'delta': 0.04270059278364587}, {'weights': [-2.5584149848484263, 1.00364
22106209202, 0.42383086467582715], 'output': 0.7790535202438367, 'delta': -0.038031325964
37354}]

```

RESULT:

Thus the Back propagation algorithm to build an Artificial Neural networks was implemented successfully.

EX. NO: 4

DATE :

**IMPLEMENTATION OF NAÏVE BAYESIAN CLASSIFIER FOR A SAMPLE
TRAINING DATASET AND TO COMPUTE ACCURACY**

AIM:

To implement Naïve Bayesian classifier for Tennis data set and to compute the accuracy with few datasets.

ALGORITHM:

1. Convert the data set into a frequency table.
2. Create likelihood table by finding the probabilities like overcast probability = 0.29 and probability of plating is 0.64.
3. Now, use Naive Bayesian equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of prediction.

Problem: Players will play if weather is sunny. Is this statement is correct?

We can solve it using above discussed method of posterior probability.

$$P(\text{Yes} | \text{Sunny}) = P(\text{Sunny} | \text{Yes}) * P(\text{Yes}) / P(\text{Sunny})$$

$$\text{Here we have } P(\text{Sunny} | \text{Yes}) = 3/9 = 0.33, P(\text{Sunny}) = 5/14 = 0.36, P(\text{Yes}) = 9/14 = 0.64$$

$$\text{Now, } P(\text{Yes} | \text{Sunny}) = 0.33 * 0.64 / 0.36 = 0.60, \text{ which has higher probability.}$$

4. Exit.

PROGRAM:

```
import pandas as pd
from sklearn import tree
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
data = pd.read_csv("E:\BALA\AI\Lab programs\pgms\Tennis.csv")
print("The first 5 values of data is :\n",data.head())
```

The first 5 values of data is :

	Outlook	Temperature	Humidity	Windy	PlayTennis
0	Sunny	Hot	High	False	No
1	Sunny	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Rainy	Mild	High	False	Yes
4	Rainy	Cool	Normal	False	Yes

obtain Train data and Train output

X = data.iloc[:, :-1]

print("\n\nThe First 5 values of train data is\n", X.head())

The First 5 values of train data is

	Outlook	Temperature	Humidity	Windy
0	Sunny	Hot	High	False
1	Sunny	Hot	High	True
2	Overcast	Hot	High	False
3	Rainy	Mild	High	False
4	Rainy	Cool	Normal	False

y = data.iloc[:, -1]

print("\n\nThe first 5 values of Train output is\n", y.head())

The first 5 values of Train output is :

0	No
1	No
2	Yes
3	Yes
4	Yes

Name: PlayTennis, dtype: object

Convert then in numbers

le_outlook = LabelEncoder()

X.Outlook = le_outlook.fit_transform(X.Outlook)

le_Temperature = LabelEncoder()

X.Temperature = le_Temperature.fit_transform(X.Temperature)

le_Humidity = LabelEncoder()

X.Humidity = le_Humidity.fit_transform(X.Humidity)

le_Windy = LabelEncoder()

X.Windy = le_Windy.fit_transform(X.Windy)

print("\n\nNow the Train data is :\n", X.head())

Now the Train data is :

	Outlook	Temperature	Humidity	Windy
0	2	1	0	0
1	2	1	0	1
2	0	1	0	0
3	1	2	0	0
4	1	0	1	0

```
le_PlayTennis = LabelEncoder()
```

```
y = le_PlayTennis.fit_transform(y)
```

```
print("\nNow the Train output is\n",y)
```

Now the Train output is

```
[0 0 1 1 1 0 1 0 1 1 1 1 1 0]
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.20)
```

```
classifier = GaussianNB()
```

```
classifier.fit(X_train,y_train)
```

```
from sklearn.metrics import accuracy_score
```

```
print("Accuracy is:",accuracy_score(classifier.predict(X_test),y_test))
```

OUTPUT:

Accuracy is: 0.6666666666666666

RESULT:

Thus the program to implement Naïve Bayesian classifier to compute the accuracy with few datasets using python was executed and verified successfully.

EX. NO: 5

DATE :

IMPLEMENTATION OF NAÏVE BAYESIAN CLASSIFIER MODEL TO CLASSIFY A SET OF DOCUMENTS AND TO MEASURE THE ACCURACY, PRECISION, AND RECALL

AIM:

To classify a set of documents using Naïve Bayesian classifier and to measure the accuracy and precision

ALGORITHM:

1. Import basic libraries.
2. Importing the dataset.
3. Data preprocessing.
4. Training the model.
5. Testing and evaluation of the model.
6. Visualizing the model.

PROGRAM:

```
from sklearn.datasets import fetch_20newsgroups

from sklearn.metrics import confusion_matrix

from sklearn.metrics import classification_report

import numpy as np

categories = ['alt.atheism', 'soc.religion.christian', 'comp.graphics', 'sci.med']

twenty_train = fetch_20newsgroups(subset='train',categories=categories,shuffle=True)

twenty_test = fetch_20newsgroups(subset='test',categories=categories,shuffle=True)

print(len(twenty_train.data))

print(len(twenty_test.data))

print(twenty_train.target_names)
```

```
print("\n".join(twenty_train.data[0].split("\n")))

print(twenty_train.target[0])
```

OUTPUT:

```
2257
1502
['alt.atheism', 'comp.graphics', 'sci.med', 'soc.religion.christian']
From: sd345@city.ac.uk (Michael Collier)
Subject: Converting images to HP LaserJet III?
Nntp-Posting-Host: hampton
Organization: The City University
Lines: 14

Does anyone know of a good way (standard PC application/PD utility) to
convert tif/img/tga files into LaserJet III format. We would also like to
do the same, converting to HPGL (HP plotter) files.

Please email any response.

Is this the correct group?

Thanks in advance. Michael.
--
Michael Collier (Programmer)
Email: M.P.Collier@uk.ac.city
Tel: 071 477-8000 x3769
Fax: 071 477-8565

The Computer Unit,
The City University,
London,
EC1V 0HB.

1
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
count_vect = CountVectorizer()
```

```
X_train_tf = count_vect.fit_transform(twenty_train.data)
```

```
from sklearn.feature_extraction.text import TfidfTransformer
```

```
tfidf_transformer = TfidfTransformer()
```

```
X_train_tfidf = tfidf_transformer.fit_transform(X_train_tf)
```

```
X_train_tfidf.shape
```

```
Out[4]: (2257, 35788)
```

```
from sklearn.naive_bayes import MultinomialNB
```

```

from sklearn.metrics import accuracy_score

from sklearn import metrics

mod = MultinomialNB()

mod.fit(X_train_tfidf, twenty_train.target)

X_test_tf = count_vect.transform(twenty_test.data)

X_test_tfidf = tfidf_transformer.transform(X_test_tf)

predicted = mod.predict(X_test_tfidf)

print("Accuracy:", accuracy_score(twenty_test.target, predicted))

print(classification_report(twenty_test.target, predicted, target_names=twenty_test.target_names))

print("confusion matrix is \n", metrics.confusion_matrix(twenty_test.target, predicted))

```

OUTPUT:

```

Accuracy: 0.8348868175765646

```

	precision	recall	f1-score	support
alt.atheism	0.97	0.60	0.74	319
comp.graphics	0.96	0.89	0.92	389
sci.med	0.97	0.81	0.88	396
soc.religion.christian	0.65	0.99	0.78	398
accuracy			0.83	1502
macro avg	0.89	0.82	0.83	1502
weighted avg	0.88	0.83	0.84	1502

```

confusion matrix is
[[192  2  6 119]
 [ 2 347  4  36]
 [ 2  11 322  61]
 [ 2  2  1 393]]

```

RESULT:

Thus the accuracy and precision was measured by Naïve Bayesian classifier model.

EX. NO: 6

DATE :

**CONSTRUCTION OF A BAYESIAN NETWORK TO DIAGNOSE CORONA
INFECTION USING STANDARD WHO DATA SET**

AIM:

To construct a Bayesian network to diagnose corona infection using WHO data set.

ALGORITHM:

This Naive Bayes is broken down into 5 parts:

- 1: Separate by Class.
- 2: Summarize Dataset.
- 3: Summarize Data by Class.
- 4: Gaussian Probability Density Function.
- 5: Class Probabilities.

PROGRAM

```
import numpy as np
import pandas as pd
from scipy.stats import randint
import pandas as pd
import matplotlib.pyplot as plt
from pandas import set_option
plt.style.use('ggplot')
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE
from sklearn.model_selection import KFold
from sklearn.model_selection import GridSearchCV
```

```

from sklearn.model_selection import RandomizedSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb
from xgboost import XGBClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import cross_val_score
from sklearn.metrics import confusion_matrix
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.feature_selection import SelectFromModel
from sklearn import metrics
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
from sklearn.metrics import classification_report
covid_19_data = pd.read_csv("E:\BALA\AI\Lab programs\pgms\covid_19_data.csv")
print(f'The shape of the dataframe is {covid_19_data.shape}')
print()
The shape of the dataframe is (306429, 8)

print(covid_19_data.info())
print()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 306429 entries, 0 to 306428
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   SNo                    306429 non-null int64
1   ObservationDate        306429 non-null object
2   Province/State        228329 non-null object
3   Country/Region        306429 non-null object
4   Last Update           306429 non-null object
5   Confirmed              306429 non-null float64
6   Deaths                306429 non-null float64
7   Recovered              306429 non-null float64
dtypes: float64(3), int64(1), object(4)
memory usage: 18.7+ MB
None

```

```

covid_19_data.replace(to_replace='?', value=np.NaN, inplace=True)
print(covid_19_data.describe(include='all'))
print()

```

	SNo	ObservationDate	Province/State	Country/Region	\
count	306429.000000	306429	228329	306429	
unique	NaN	494	737	229	
top	NaN	05/29/2021	Unknown	Russia	
freq	NaN	765	4123	30251	
mean	153215.000000	NaN	NaN	NaN	
std	88458.577156	NaN	NaN	NaN	
min	1.000000	NaN	NaN	NaN	
25%	76608.000000	NaN	NaN	NaN	
50%	153215.000000	NaN	NaN	NaN	
75%	229822.000000	NaN	NaN	NaN	
max	306429.000000	NaN	NaN	NaN	

	Last Update	Confirmed	Deaths	Recovered
count	306429	3.064290e+05	306429.000000	3.064290e+05
unique	1905	NaN	NaN	NaN
top	2021-04-02 15:13:53	NaN	NaN	NaN
freq	239885	NaN	NaN	NaN
mean	NaN	8.567091e+04	2036.403268	5.042029e+04
std	NaN	2.775516e+05	6410.938048	2.015124e+05
min	NaN	-3.028440e+05	-178.000000	-8.544050e+05
25%	NaN	1.042000e+03	13.000000	1.100000e+01
50%	NaN	1.037500e+04	192.000000	1.751000e+03
75%	NaN	5.075200e+04	1322.000000	2.027000e+04
max	NaN	5.863138e+06	112385.000000	6.399531e+06

```
print(covid_19_data['Country/Region'].value_counts())
```

```
print(covid_19_data.isnull().sum())
```

```
Russia          30251
US              26740
Japan           18059
Mainland China  15758
India           13182
...
Azerbaijan      1
North Ireland    1
Republic of Ireland 1
Cape Verde       1
East Timor       1
Name: Country/Region, Length: 229, dtype: int64
SNo              0
ObservationDate  0
Province/State   78100
Country/Region   0
Last Update      0
Confirmed        0
Deaths           0
Recovered        0
dtype: int64
```

```
import seaborn as sns
```

```
sns.countplot(x='Country/Region', data=covid_19_data, linewidth=3)
```

```
plt.show()
```

```
covid_19_data[['ObservationDate', 'Province/State', 'Country/Region', 'Last Update', 'Confirmed', 'Deaths', 'Recovered']].hist(bins=50, figsize=(15,8))
```

```
plt.show()
```

```
covid_19_data['Country/Region'].fillna(covid_19_data['Country/Region'].mode()[0], inplace=True)
```

```
covid_19_data['Confirmed'].fillna(covid_19_data['Confirmed'].mode()[0], inplace=True)
```

```
X = covid_19_data.drop(['Deaths'], axis=1)
```

```
y = covid_19_data.Recovered
```

```
X=X[['confirmed', 'Recovered']]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

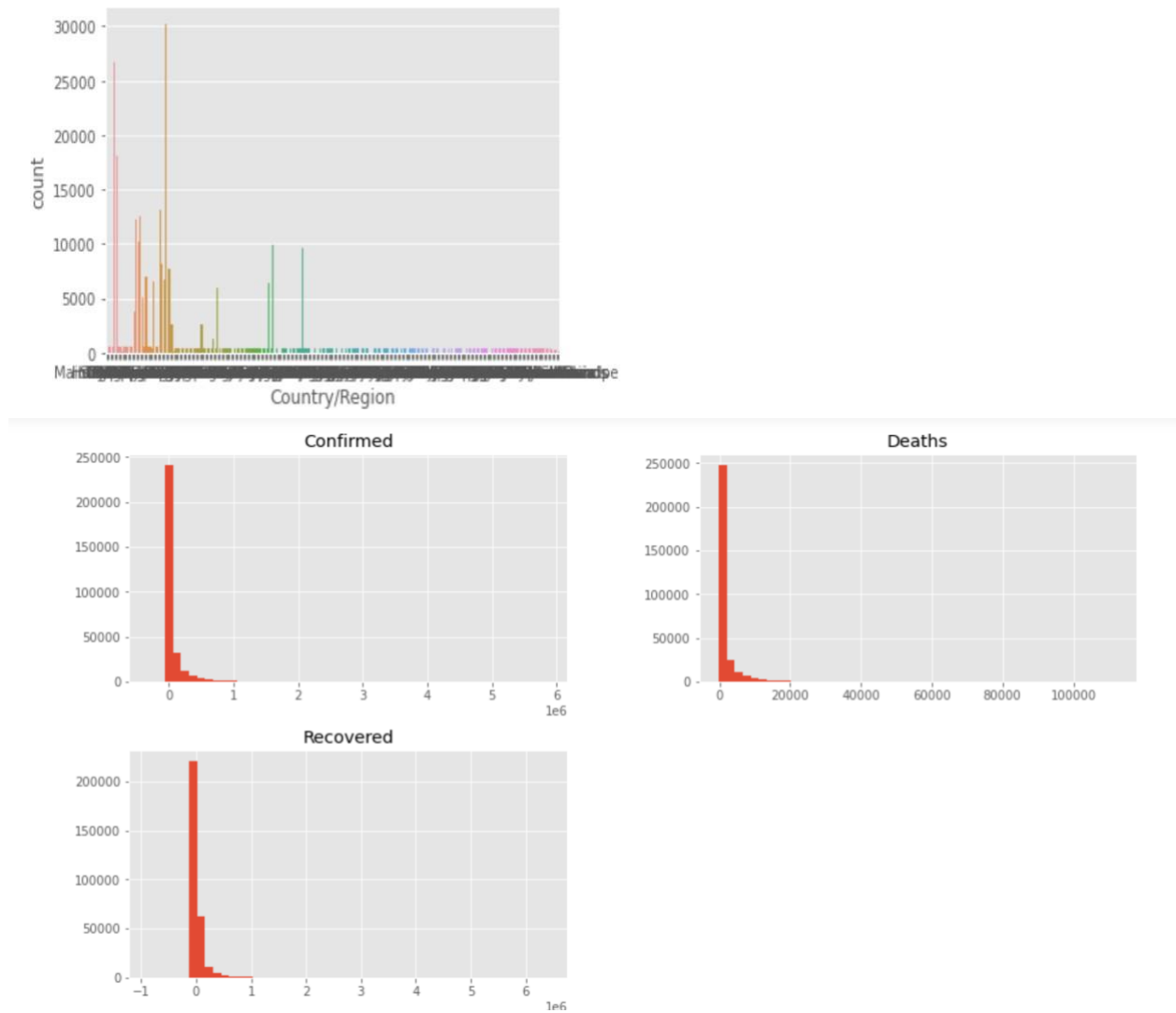
```
NB_classifier = GaussianNB()
```

```
NB_classifier.fit(X_train, y_train)
```

```
y_predict = NB_classifier.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_predict)
sns.heatmap(cm, annot=True, cmap='Blues')
print(classification_report(y_test, y_predict))
```

OUTPUT:



RESULT:

Thus the program to diagnose corona infection using Bayesian network was successfully implemented using python.

EX. NO: 7

DATE :

**COMPARISON OF CLUSTERING IN EM ALGORITHM AND K-MEANS
ALGORITHM USING THE SAME DATA SETS**

AIM:

To compare the clustering in EM algorithm and K-means algorithm using the same data sets.

ALGORITHM:

The K-means implementation is as follows:

1. Choose the number of clusters k.
2. Select k random points from the data as centroids.
3. Assign all the points to the closest cluster centroid.
4. Recompute the centroids of newly formed clusters.
5. Repeat steps 3 and 4.

The EM implementation is as follows:

1. **Expectation step (E - step):** It involves the estimation (guess) of all missing values in the dataset so that after completing this step, there should not be any missing value.
2. **Maximization step (M - step):** This step involves the use of estimated data in the E-step and updating the parameters.
3. **Repeat E-step and M-step** until the convergence of the values occurs.

PROGRAM:

```
from sklearn.cluster import KMeans
from sklearn import preprocessing
from sklearn.mixture import GaussianMixture
from sklearn.datasets import load_iris
import sklearn.metrics as sm
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```

dataset=load_iris()
# print(dataset)

X=pd.DataFrame(dataset.data)
X.columns=['Sepal_Length','Sepal_Width','Petal_Length','Petal_Width']
y=pd.DataFrame(dataset.target)
y.columns=['Targets']
# print(X)

plt.figure(figsize=(14,7))
colormap=np.array(['red','lime','black'])

# REAL PLOT
plt.subplot(1,3,1)
plt.scatter(X.Petal_Length,X.Petal_Width,c=colormap[y.Targets],s=40)
plt.title('Real')

# K-PLOT
plt.subplot(1,3,2)
model=KMeans(n_clusters=3)
model.fit(X)
predY=np.choose(model.labels_,[0,1,2]).astype(np.int64)
plt.scatter(X.Petal_Length,X.Petal_Width,c=colormap[predY],s=40)
plt.title('KMeans')

# GMM PLOT
scaler=preprocessing.StandardScaler()
scaler.fit(X)

```

```

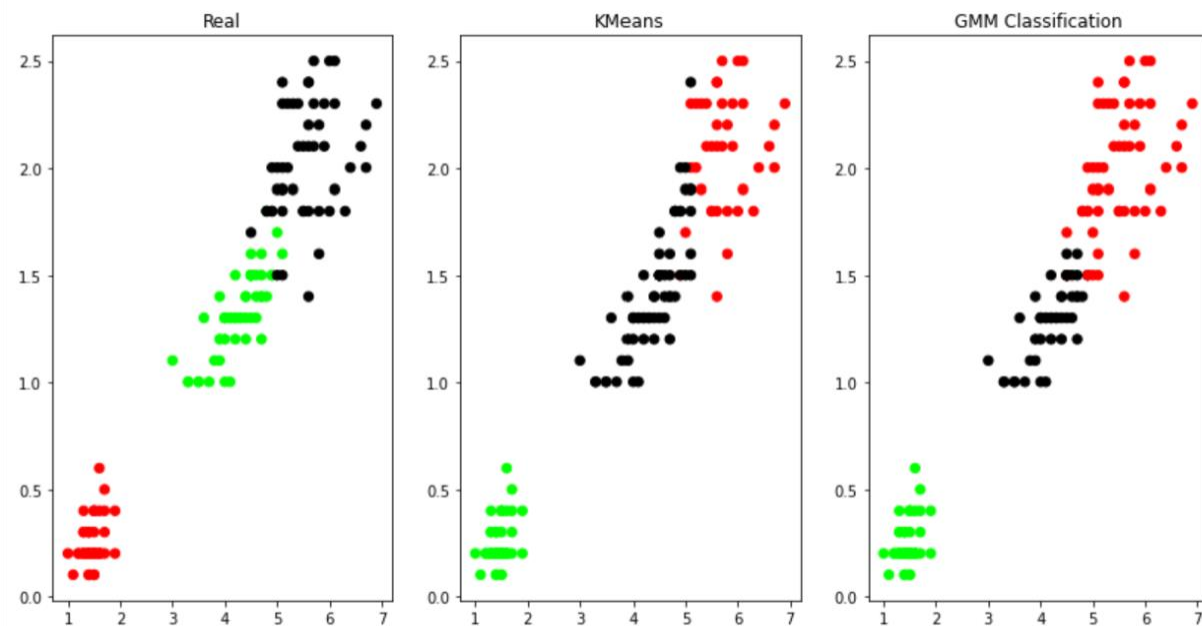
xsa=scaler.transform(X)
xs=pd.DataFrame(xsa,columns=X.columns)
gmm=GaussianMixture(n_components=3)
gmm.fit(xs)
y_cluster_gmm=gmm.predict(xs)
plt.subplot(1,3,3)
plt.scatter(X.Petal_Length,X.Petal_Width,c=colormap[y_cluster_gmm],s=40)

plt.title('GMM Classification')

```

OUTPUT:

Text(0.5, 1.0, 'GMM Classification')



RESULT:

Thus the program to compare clustering in EM and K-means algorithm with few datasets was performed successfully.

EX. NO: 8

DATE :

**IMPLEMENTATION OF K-NEAREST NEIGHBOUR ALGORITHM TO CLASSIFY
THE IRIS DATA SET**

AIM:

To implement K-Nearest Neighbour algorithm to classify iris data set.

ALGORITHM:

1. Calculate the Information Gain of each feature.
2. Considering that all rows don't belong to the same class, split the dataset **S** into subsets using the feature for which the Information Gain is maximum.
3. Make a decision tree node using the feature with the maximum Information gain.
4. If all rows belong to the same class, make the current node as a leaf node with the class as its label.
5. Repeat for the remaining features until we run out of all features, or the decision tree has all leaf nodes.

PROGRAM:

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
import numpy as np
dataset=load_iris()
X_train,X_test,y_train,y_test=train_test_split(dataset["data"],dataset["target"],random_state=0)
kn=KNeighborsClassifier(n_neighbors=1)
kn.fit(X_train,y_train)
for i in range(len(X_test)):
```

```

x=X_test[i]

x_new=np.array([x])

prediction=kn.predict(x_new)

print("TARGET=",y_test[i],dataset["target_names"]
[y_test[i]],"PREDICTED=",prediction,dataset["target_names"][prediction])

print(kn.score(X_test,y_test))

```

OUTPUT:

```

TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']

```

```
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 2 virginica PREDICTED= [2] ['virginica']
TARGET= 1 versicolor PREDICTED= [1] ['versicolor']
TARGET= 0 setosa PREDICTED= [0] ['setosa']
TARGET= 1 versicolor PREDICTED= [2] ['virginica']
0.9736842105263158
```

RESULT:

Thus the program for K-Nearest Neighbour algorithm was implemented successfully using an iris data set.

EX. NO: 9

DATE :

**IMPLEMENTATION OF THE NON-PARAMETRIC LOCALLY WEIGHTED
REGRESSION ALGORITHM IN ORDER TO FIT DATA POINTS**

AIM:

To implement the non-parametric Locally Weighted Regression algorithm in order to fit data points.

ALGORITHM:

1. Read the Given data Sample to X and the curve (linear or non linear) to Y
2. Set the value for Smoothing parameter or Free parameter say τ
3. Set the bias /Point of interest set x_0 which is a subset of X
4. Determine the weight matrix using :

$$w(x, x_0) = e^{-\frac{(x-x_0)^2}{2\tau^2}}$$

5. Determine the value of model term parameter β using:

$$\hat{\beta}(x_0) = (X^T W X)^{-1} X^T W y$$

6. Prediction = $x_0 * \beta$

PROGRAM:

```
from math import ceil
import numpy as np
from scipy import linalg
def lowess(x, y, f, iterations):
    n = len(x)
    r = int(ceil(f * n))
    h = [np.sort(np.abs(x - x[i]))[r] for i in range(n)]
```

```

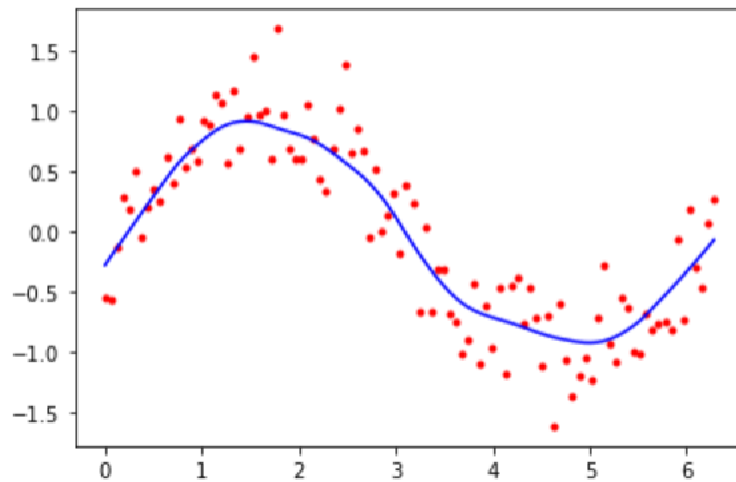
w = np.clip(np.abs((x[:, None] - x[None, :]) / h), 0.0, 1.0)
w = (1 - w ** 3) ** 3
yest = np.zeros(n)
delta = np.ones(n)
for iteration in range(iterations):
    for i in range(n):
        weights = delta * w[:, i]
        b = np.array([np.sum(weights * y), np.sum(weights * y * x)])
        A = np.array([[np.sum(weights), np.sum(weights * x)], [np.sum(weights * x),
np.sum(weights * x * x)]])
        beta = linalg.solve(A, b)
        yest[i] = beta[0] + beta[1] * x[i]
    residuals = y - yest
    s = np.median(np.abs(residuals))
    delta = np.clip(residuals / (6.0 * s), -1, 1)
    delta = (1 - delta ** 2) ** 2
return yest

import math
n = 100
x = np.linspace(0, 2 * math.pi, n)
y = np.sin(x) + 0.3 * np.random.randn(n)
f = 0.25
iterations = 3
yest = lowess(x, y, f, iterations)
import matplotlib.pyplot as plt
plt.plot(x, y, "r.")
plt.plot(x, yest, "b-")

```

OUTPUT:

```
[<matplotlib.lines.Line2D at 0x1a226401070>]
```

**RESULT:**

Thus the non-parametric Locally Weighted Regression algorithm to fit data points was implemented successfully.